

DATA 266 Lab 1 — Team 32 Report

Team members: Shreya Akotiya (shreya_akotiya), Zoheb Waghu (zoheb_waghu)

Repository: <https://github.com/sakotiya/data266-lab1>

Team ownership statement

Each member independently designed, implemented, trained and evaluated their own models for every task, in their own folder (<task>/shreya_akotiya/, <task>/zoheb_waghu/). For Task 1, Shreya built and trained the 12-layer deep-narrow character GPT, and Zoheb built and trained the 4-layer shallow-wide character GPT. The two implementations are separate code bases (Shreya: a config-driven notebook; Zoheb: `src/` modules plus a notebook). The team agreed on the shared decisions before training: the reading of "100K / 10K" as sequences, disjoint data slices, a vocabulary built from the train split only, and a common `metrics_report.csv` schema so the numbers can be placed side by side. The comparison and analysis below were written jointly.

Task 1 — GPT-style character language model from scratch

1.1 Shared setup

Both models follow the same rules, so differences between them come from the design choices in §1.2.

- **Data.** TinyStories, character-level. Both members read the same raw file at **different, non-overlapping offsets**: Shreya chars [0, 34M), Zoheb chars from 34M. Each member therefore has their own train/validation split.
- **Split size.** "Training (100K) and validation (10K)" is read as **sequences** of 256 characters: 100,000 train sequences (25.6M characters) and 10,000 validation sequences (2.56M characters). Targets are inputs shifted by one.
- **Vocabulary.** Own `char_to_idx / idx_to_char`, built from the training split only, with an <unk> slot for unseen characters.
- **No prebuilt Transformer modules.** Attention (Q/K/V projections, scaled dot product, causal mask, softmax, head split/merge), LayerNorm placement, feed-forward, residuals, token and positional embeddings and the LM head are all hand-written. Neither code base uses `nn.Transformer*`, `nn.MultiheadAttention` or fused scaled-dot-product attention.
- **Causal mask verified behaviourally.** Both members changed the token at position `j` and confirmed that the logits at every position before `j` are bit-identical, while logits from `j` onward change.
- **Training.** Cross-entropy loss, AdamW ($\beta = 0.9, 0.95$, weight decay 0.1), linear warm-up followed by cosine decay, gradient clipping at 1.0, 10 epochs.
- **Generation.** Greedy decoding and temperature sampling from the prompt "Once upon a time".

1.2 Model comparison — architecture and hyperparameters

The two models are the team's depth-vs-width comparison: a deep-narrow model against a shallow-wide one at the same embedding width and the same number of training tokens.

	shreya_akotiya	zoheb_waghu
Design idea	deep-narrow	shallow-wide
Transformer blocks	12	4
Attention heads (dims/head)	8 (32)	4 (64)
Embedding width d_model	256	256
Feed-forward width	1024 (4x)	1024 (4x), GELU
Normalisation	pre-LN	pre-LN
Positional encoding	learned	learned
Context length	256	256
Vocabulary size	78	97
Dropout	0.05	0.1
Parameters	9,570,816	3,274,752
Batch size (tokens/step)	64 (16,384)	32 (8,192)
Steps (per epoch / total)	1,562 / 15,620	3,125 / 31,250
Peak → min learning rate	2.5e-4 → 2.5e-5	3e-4 → 3e-5
Warm-up	781 steps (5%)	1,000 steps (~3%)
Epochs	10	10
Training tokens seen	256M	256M
Seed	1337	1337
Hardware	NVIDIA Tesla T4 16 GB (Google Colab), CUDA, PyTorch 2.11	NVIDIA GeForce RTX 4090 24 GB, AMD Ryzen 9 7950X, CUDA, PyTorch 2.5.1

The two models also differ in dropout, batch size, learning rate and head size, not only in depth. The comparison is therefore **deep-narrow vs shallow-wide as complete designs**, not a controlled ablation of depth alone.

1.3 Metrics — side by side

All numbers are from each member's `metrics_report.csv` (team schema) and `metrics_report_extended.csv`. Cross-entropy is in nats per character on each member's own 10K-sequence validation split.

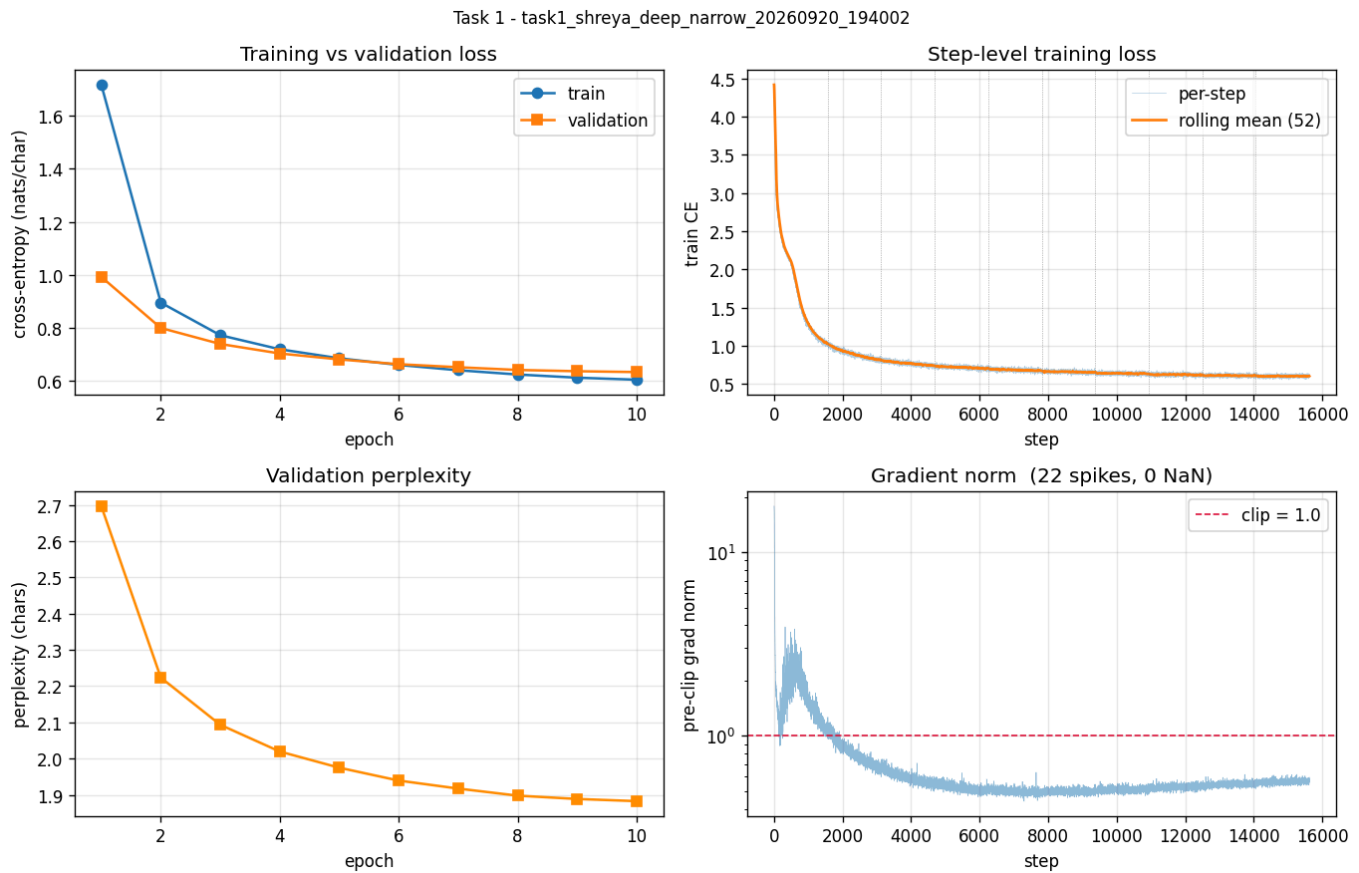
Metric	shreya_akotiya	zoheb_waghu
Training cross-entropy (final epoch)	0.6038	0.6982
Validation cross-entropy (best)	0.6327	0.7019
Perplexity	1.883	2.018
Bits-per-character	0.913	1.013
Generalization gap (val – train)	0.0289	0.0037
Top-1 next-character accuracy	79.9%	77.7%
Distinct-1 / 2 / 3 (character) ‡	0.0029 / 0.0286 / 0.1247	0.0125 / 0.0997 / 0.2832
Repeated 4-gram rate (character) ‡	0.721	0.559
Gradient norm mean / max	0.687 / 17.79	0.744 / 9.77
Loss spikes †	22	0
NaN / Inf events	0	0
Parameter count	9,570,816	3,274,752
Training throughput (tokens/sec) ¶	24,209 (T4)	663,173 (RTX 4090)
Generation throughput (tokens/sec) ¶	87.6	507.6
Peak GPU memory (torch.cuda.max_memory_allocated)	8.06 GB	0.99 GB
Total training time ¶	176.2 min	6.4 min
Best epoch	10 (last)	10 (last)

Comparability notes

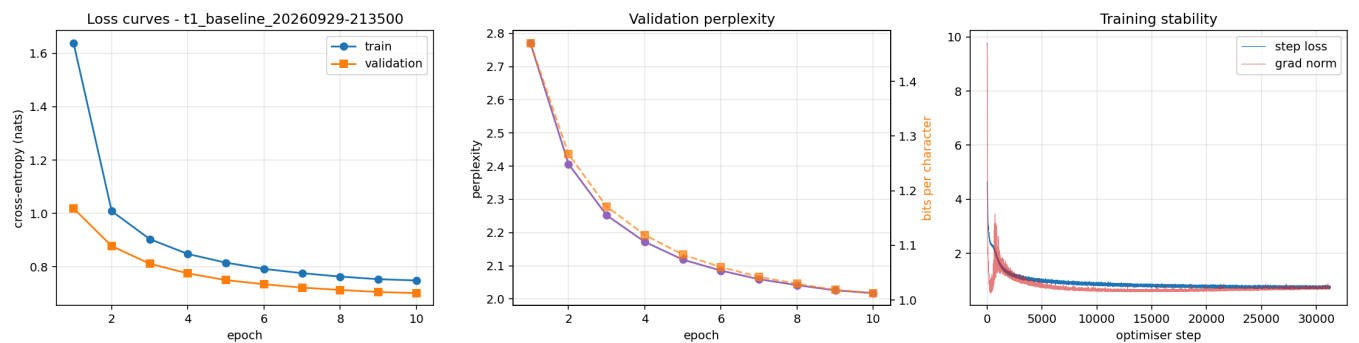
- ‡ **Diversity metrics are not directly comparable.** Both use the same formula (unique n-grams ÷ total n-grams over characters), but Shreya's are measured on 20,000 generated characters at T = 1.0 and Zoheb's on ~3,600 characters pooled over greedy, T = 0.8 and T = 1.2. Distinct-n falls and repeated-n-gram rate rises as the text gets longer, so the gap between the columns mostly reflects sample length, not model quality.
- † **Loss spikes use different detectors.** Shreya flags a step whose loss exceeds the rolling mean by 3σ (100-step window); her 22 spikes are each about 7% above the local mean. Zoheb flags a loss above 2× the running median, a much looser threshold that none of Shreya's spikes would meet. Both runs are stable by either definition: no NaNs, and validation loss improved every epoch.
- ¶ **Speed is not comparable.** Shreya trained on a Tesla T4 and Zoheb on an RTX 4090, a much faster GPU, so throughput and training time mostly reflect the hardware, not the models.
- **Peak memory is comparable.** Both runs measure it with `torch.cuda.max_memory_allocated`. The 8× difference comes from the deeper model (12 vs 4 blocks of stored activations) and the larger batch (64 vs 32).
- The two validation sets are different slices of TinyStories, so the loss comparison is close but not paired.

1.4 Training curves

shreya_akotiya (12 × 256):



zoheb_waghu (4 × 256):



Validation loss per epoch:

Epoch	1	2	3	4	5	6	7	8	9	10
shreya — val bpc	1.431	1.153	1.066	1.014	0.982	0.956	0.939	0.924	0.917	0.913
zoheb — val bpc	1.470	1.267	1.171	1.119	1.083	1.060	1.042	1.030	1.019	1.013

Both curves fall at every epoch and both models reach their best validation loss at the last epoch. The deeper model is ahead from epoch 1, and the gap between the two models grows from 0.04 bpc at epoch 1 to about 0.10 bpc by epoch 10.

1.5 Joint analysis

What the comparison shows

1. **Depth improved modelling quality.** At the same width and the same 256M training tokens, the 12-block model reaches 0.913 bits per character against 1.013 (about 10% lower) and 79.9% next-character accuracy against 77.7%. A character model has to compose characters into words, words into phrases and phrases into sentences, and each block is one more step of that composition.
2. **The cost is size and memory.** The deep model has 2.9× the parameters and needs 8× the peak GPU memory, and its 12 blocks must run one after another, so each step does about three times the work of the 4-block model. The training times (176 min vs 6 min) are not a fair measure of this cost because the GPUs differ.
3. **The shallow model barely overfits.** Its generalization gap is 0.004 against 0.029. That reflects its smaller capacity rather than better regularisation: it fits both train and validation less well.
4. **Both models are under-trained.** Validation loss was still falling at epoch 10 in both runs, and in both the best checkpoint is the last. Ten epochs was a budget limit, not convergence.
5. **Training was stable for both.** No NaN or Inf events, and mean gradient norms of 0.69 and 0.74, below the clipping threshold of 1.0. Shreya's largest gradient norm (17.8) is at initialisation; after warm-up her maximum is 2.9, and no step after 1,910 needed clipping.

Strengths

- Both implementations pass a behavioural causal-mask test, not only a visual one.
- Both produce fluent, correctly spelled English at low temperature, with correct word boundaries, punctuation, quotation marks and the typical TinyStories structure.
- Disjoint data slices give each member a genuinely separate train/validation split.

Weaknesses and shared failure modes

The two models fail in the same ways, which suggests the failures come from the shared setup (character-level next-token prediction, 256-character context, small model, no decoding constraints) rather than from either design.

Failure type	shreya_akotiya example	zoheb_waghu example
Repetition under greedy decoding	repeats "Once upon a time, there was a little girl named Lily."	"The box was so happy and the box was so happy to see the box."
Semantic contradiction / impossible meaning	"Lily was happy to hear that her mom was sad."	"Her pocket seemed sad to melt."
Losing track of who is who	"Lily didn't want to share his toy friends"	restarts a new story mid-generation ("Once upon a time...")
Non-words at high temperature	"strets", "pumpins", "knowled" (3.0% non-words at T = 1.2)	"aboven", "MPleaf", "spreadying" at T = 1.2

Temperature trades one failure for another in both models. Greedy and low temperature give correct spelling but loop; high temperature removes the loops but breaks spelling and meaning. In Shreya's temperature sweep, $T = 0.8$ was the best balance (0.42% non-words, no repeated phrases). Zoheb's repeated-4-gram rate falls from 0.61 (greedy) to 0.37 ($T = 0.8$) to 0.23 ($T = 1.2$), while his semantic errors grow at $T = 1.2$.

Limitations

- Single seed per model, so there is no variance estimate.
- The two models differ in several hyperparameters at once (see §1.2), so the quality gap cannot be attributed to depth alone.
- The diversity and loss-spike metrics were computed with different sample sizes and detectors (see §1.3).
- The 256-character context is shorter than most stories, so neither model can keep a story consistent from start to finish.
- Generation uses no KV cache, so throughput figures understate what the models could do.

What the team would try next

1. **Train longer.** Both curves were still falling. We expect 15–20 epochs to lower validation loss further for both, with the deep model's gap growing first.
2. **Controlled depth ablation.** Train 256×4 and 256×12 with the same batch size, learning rate, dropout and seed, to isolate the effect of depth.
3. **Top-k / nucleus sampling ($p \approx 0.9$).** Cutting the low-probability tail should keep the diversity of higher temperatures while avoiding their non-words, and should break greedy loops.
4. **Longer context (512–1024)** and masking at story boundaries, targeting the coherence and story-restart failures that decoding changes cannot fix.
5. **Unified evaluation script.** Re-score both models' diversity on the same number of characters at the same temperature, with one loss-spike definition.

1.6 Individual failure analyses

Full write-ups with verbatim snippets are in each member's folder:

- task1_llm/shreya_akotiya/failure_analysis.md:
 1. semantic contradiction (greedy);
 2. pronoun and reference confusion ($T = 0.5$);
 3. non-words at high temperature ($T = 1.0$);
 4. the temperature trade-off across greedy to $T = 1.2$.
- task1_llm/zoheb_waghu/failure_analysis.md:
 1. phrase-level repetition loop (greedy);
 2. invalid words and semantic breakdown at high temperature ($T = 1.2$);
 3. story-boundary confusion: restarting a new story mid-generation (greedy).

1.7 Evidence

	shreya_akotiya	zoheb_waghu
Run ID	task1_shreya_deep_narrow_20260920_194002	t1_baseline_20260929-213500
Config	task1_llm/shreya_akotiya/config.yaml	task1_llm/zoheb_waghu/configs/gpt_base_line.yaml
Checkpoint	task1_llm/shreya_akotiya/checkpoints/task1_shreya_deep_narrow_20260920_194002_best.pt (sha256 f540786b...)	task1_llm/zoheb_waghu/checkpoints/t1_baseline_20260929-213500_best.pt (sha256 6b7ecfe1...)
Raw log	reproducibility/raw_logs/shreya_akotiya/task1_llm/task1_shreya_deep_narrow_20260920_194002.log	reproducibility/raw_logs/zoheb_waghu/task1_llm/t1_baseline_20260929-213500.log (+.jsonl)
Manifest	reproducibility/manifests/shreya_akotiya/task1_shreya_deep_narrow_20260920_194002.json	reproducibility/manifests/zoheb_waghu/task1_llm_manifest.md
Loss curves	outputs/plots/loss_curves.png	outputs/plots/loss_curves_t1_baseline_20260929-213500.png
Samples	outputs/samples/{greedy,T0.5,T0.8,T1.0,T1.2}.txt	outputs/samples/samples_t1_baseline_20260929-213500.txt
Causal-mask check	outputs/plots/causal_mask_check.png	src/evaluate.py::causal_mask_probe (logged in run)
Metrics	metrics_report.csv, metrics_report_extended.csv	metrics_report.csv, metrics_report_extended.csv

References

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems 30.
2. Eldan, R., & Li, Y. (2023). *TinyStories: How Small Can Language Models Be and Still Speak Coherent English?* arXiv:2305.07759.